The 15 RAG concepts every AI engineer should know

A plain English field guide for working software engineers. No math degree required. No prior ML background assumed.

Retrieval Augmented Generation, or RAG, is the pattern behind almost every AI product that answers questions about private data. Support bots that know your docs. Internal search that actually works. Legal and medical assistants that cite sources.

The idea is simple. A language model only knows what was in its training data. It has never seen your company wiki, your product manual, or last quarter's tickets. So instead of retraining the model, you go find the relevant text yourself, paste it into the prompt, and ask the model to answer using only that.

Retrieval is the hard part. The model is the easy part. That is the whole insight. Ninety percent of the work in a RAG system is a search problem, a data pipeline problem, and a latency and cost problem. Those are things you have already been solving for years.

Which is why so many engineers with 2 to 6 years of production experience are landing AI engineering roles. You are not behind. You are unspecialized. This guide closes the vocabulary gap.

HOW TO READ THIS GUIDE

- **What it is** in plain language, then what it is actually for.
- **When to reach for it** so you know the trigger, not just the term.
- **Background** so you understand why the concept exists at all.
- **How to learn it** with a build task you can finish in an afternoon.
- **Two meters.** Impact is how much the concept moves the quality of your system. Hiring signal is how often it shows up in real AI engineering job descriptions and interviews.

01	**Chunking strategy** How you cut long documents into smaller pieces before storing them.

THE PURPOSE

You cannot paste a 400 page handbook into a prompt. Even if you could afford it, the model answers worse when buried in noise. So you slice documents into passages small enough to be precise and large enough to make sense on their own.

WHEN TO USE IT

Always. This is step one of every RAG pipeline, and it is the single most common cause of bad results. If your bot gives vague or wrong answers, look here before you touch the model.

BACKGROUND

The naive approach is fixed size, for example every 500 characters. It is fast and it is usually wrong, because it slices mid sentence and mid table. Better approaches respect structure: split on headings, paragraphs, markdown sections, code blocks, or function boundaries. Think of it exactly like designing a database schema. Get the shape wrong and everything downstream suffers.

HOW TO LEARN IT

Take one real PDF or one repo README. Chunk it three ways: fixed 500 characters, split by paragraph, split by heading. Ask the same five questions against each. Read what comes back. You will feel the difference in twenty minutes, and that intuition is worth more than any blog post.

IMPACT ● ● ● ● ●

Highest leverage decision in the whole pipeline.

HIRING SIGNAL ● ● ● ● ●

Near universal. Expect to defend your choice out loud.

02

Chunk overlap

Letting the end of one chunk repeat at the start of the next.

THE PURPOSE

Insurance against a bad cut. If the answer to a question straddles the boundary between chunk 4 and chunk 5, neither chunk contains the full thought. Repeating a bit of text on both sides means at least one chunk holds the complete idea.

WHEN TO USE IT

Whenever you are chunking by size rather than by structure. If you split cleanly on headings, you need very little overlap. If you split on character count, you need it.

BACKGROUND

The usual range is 10 to 20 percent of chunk size. There is a real tradeoff. More overlap means better recall but a bigger index, higher storage cost, and duplicate passages competing for the same slot in your prompt. Engineers who crank overlap to 50 percent to be safe end up with a system that returns the same paragraph four times.

HOW TO LEARN IT

Index the same document at 0 percent overlap and 20 percent overlap. Write one question whose answer spans a chunk boundary on purpose. Watch the zero overlap version fail. That is the entire lesson.

IMPACT ● ● ● ● ○

Cheap fix for a common failure mode.

HIRING SIGNAL ● ● ● ○ ○

Rarely asked alone. Comes up inside chunking questions.

03 Embeddings

Turning a piece of text into a list of numbers that captures its meaning.

THE PURPOSE

Computers cannot compare meaning directly. An embedding model reads a chunk of text and outputs an array of numbers, often 768 or 1536 of them. Text with similar meaning produces similar arrays. That is what makes “how do I reset my password” match a document titled “Credential recovery procedure” even though they share no words.

WHEN TO USE IT

Every time you index a document and every time a user asks a question. Both sides get embedded, then compared.

BACKGROUND

You do not train these. You call an API or run an open model off the shelf. The practical decisions are which model, what dimension, and what it costs at your document volume. Two rules that save people weeks: the query and the documents must use the same embedding model, and changing embedding models means reindexing everything from scratch. Treat that choice like picking a database.

HOW TO LEARN IT

Embed five sentences. Two about dogs, two about finance, one about a golden retriever's vet bill. Print the similarity scores between all pairs. Seeing that last sentence land between the two clusters is the moment the concept stops being abstract.

IMPACT ● ● ● ● ●
Foundational. Nothing else works without it.

HIRING SIGNAL ● ● ● ● ●
Assumed knowledge. Not knowing this ends the interview.

04

Vector databases

Where those arrays of numbers live, and how you search them fast.

THE PURPOSE

Comparing your question against every chunk one at a time is fine for a thousand chunks and unusable for ten million. A vector database indexes the numbers so it can find the closest matches in milliseconds without checking all of them.

WHEN TO USE IT

Once your corpus outgrows memory or you need filtering, updates, and persistence. Under a few thousand chunks, a plain array and a loop is genuinely fine, and saying so in an interview reads as senior judgment rather than ignorance.

BACKGROUND

Options split into dedicated services like Pinecone, Weaviate, Qdrant, and Chroma, and extensions to databases you already run, like pgvector for Postgres. Most teams should start with pgvector because it means one less system to operate. Under the hood these use approximate nearest neighbor indexes such as HNSW, which trade a tiny bit of accuracy for enormous speed. You do not need to implement HNSW. You do need to know that "approximate" means occasionally imperfect, and that this is a deliberate tradeoff you control.

HOW TO LEARN IT

Build the same tiny retriever twice: once with a Python list and a similarity loop, once with pgvector or Chroma. You will understand what the database is actually doing for you, which is the difference between using a tool and being able to debug it.

IMPACT ● ● ● ● ○
Matters at scale. Overkill on day one.

HIRING SIGNAL ● ● ● ● ●
Named explicitly in most job postings.

05

Similarity search

The math that decides which chunks are closest to the question.

THE PURPOSE

Once everything is numbers, “relevance” becomes distance. Cosine similarity, the most common measure, asks whether two arrays point in the same direction. Score near 1 means very similar. Near 0 means unrelated.

WHEN TO USE IT

It is running under every retrieval call you make. You rarely write it yourself. You do read its output constantly while debugging.

BACKGROUND

The trap is treating the score as a truth value. A top result with a score of 0.71 might be excellent in one corpus and garbage in another, because scores are only meaningful relative to your own data. Teams that hardcode “reject anything under 0.8” without measuring their own distribution ship systems that silently refuse to answer. Look at your actual score histogram before you pick a threshold.

HOW TO LEARN IT

Implement cosine similarity in about four lines of numpy, then confirm it matches what your vector store returns. Ten minutes of work, permanent removal of the mystery.

IMPACT ● ● ● ○ ○

Mostly handled for you, but you must read the scores.

HIRING SIGNAL ● ● ● ● ○

Common warmup question. Easy points if you can explain it simply.

06

Top k

How many chunks you pull back and hand to the model.

THE PURPOSE

Your retriever ranks every chunk by closeness to the question. The letter k is just where you cut the list. Set k to 5 and the five closest chunks go into the prompt. It is a knob, not a theory.

WHEN TO USE IT

You tune it the moment your answers start feeling either thin or unfocused. Too low and the right passage never makes it in. Too high and you pay for tokens, add latency, and bury the good chunk among four mediocre ones.

BACKGROUND

Typical values run 3 to 10 for direct question answering, and higher when you are reranking afterward. A very common production pattern is to retrieve wide and then narrow: pull 50 candidates cheaply, then use a reranker to pick the best 5. That gets you recall and precision instead of forcing a choice between them.

HOW TO LEARN IT

Run the same ten questions at k equals 1, 3, 5, and 20 and log both answer quality and token count. You will find your own sweet spot in one sitting, and now you have a chart to show in an interview.

IMPACT ● ● ● ● ○

Direct lever on quality, cost, and speed at once.

HIRING SIGNAL ● ● ● ○ ○

Shows up as “how did you tune it” rather than “define it”.

07

Hybrid search

Running keyword search and meaning search together, then merging the results.

THE PURPOSE

Embeddings are great at meaning and bad at exact strings. Ask about error code ERR_4021 or part number XJ 900 and semantic search will happily return something about errors in general. Old fashioned keyword search nails exact matches and misses paraphrases. Run both and you cover each other's blind spots.

WHEN TO USE IT

Any domain full of identifiers: product SKUs, ticket numbers, function names, drug names, legal citations, internal acronyms. In practice that is most enterprise use cases.

BACKGROUND

The keyword half is usually BM25, a ranking function that has been the backbone of search engines for decades. Merging the two result lists is typically done with Reciprocal Rank Fusion, which sounds intimidating and is roughly ten lines of code that rewards documents ranked highly by either method. This is one of the highest return upgrades available, and a lot of teams never do it because it feels less exciting than swapping models.

HOW TO LEARN IT

Add BM25 alongside your existing vector search, fuse the rankings, and test with queries containing exact codes. Watching semantic-only

search fail on an error code and hybrid nail it is the demo that makes hiring managers lean forward.

IMPACT ● ● ● ● ●
Often the biggest single quality jump available.

HIRING SIGNAL ● ● ● ● ○
Strong separator between hobby projects and production work.

08

Metadata filtering

Narrowing the search to only the documents that are allowed or relevant.

THE PURPOSE

Store attributes alongside each chunk: author, department, date, customer ID, document type, access level. Then constrain the search. This is a WHERE clause on your retrieval, and it is the difference between a toy and something legal will approve.

WHEN TO USE IT

Multi tenant products, permissions, versioned documentation, anything time sensitive. If two customers share one index and you have no filter, you have a data leak, not a feature.

BACKGROUND

This is where your existing backend experience becomes an unfair advantage. Access control, tenancy, and index design are things you already understand. The subtlety specific to vector search is pre filter versus post filter. Filtering after retrieval can leave you with two results when you asked for ten, because the filter threw the rest away. Filtering before retrieval is usually correct and most vector stores support it natively.

HOW TO LEARN IT

Index documents for two fake customers in one collection. Write a query that returns the wrong customer's data. Then fix it with a filter. Now you have a war story about a security bug you caught, which interviews far better than a feature you built.

IMPACT ● ● ● ● ●
Required for anything real. Not optional.

HIRING SIGNAL ● ● ● ● ○
Signals you have thought about production, not demos.

09

Query rewriting

Cleaning up the user's question before you search with it.

THE PURPOSE

Real users type badly. They write “does it work with mine” after three previous messages. That string retrieves nothing useful. A small model call rewrites it into “does the Pro plan support SAML single sign on” and suddenly retrieval works.

WHEN TO USE IT

Any chat interface, because follow up questions depend on earlier turns. Also useful for expanding acronyms and generating a few query variations to search in parallel.

BACKGROUND

The two main flavors are rewriting for context, which folds conversation history into a standalone question, and expansion, which fires several phrasings and merges the results. The cost is an extra model call before every search, so use a small fast model and cache aggressively. This is the point where RAG stops being one function call and becomes a pipeline, which is exactly the kind of system your backend instincts are built for.

HOW TO LEARN IT

Log ten real questions from any support inbox. Search with them raw. Rewrite each by hand, search again, compare. Then automate the rewrite you were doing manually.

IMPACT ● ● ● ● ○

Transformative for chat. Optional for single shot search.

HIRING SIGNAL ● ● ● ○ ○

Comes up in system design rounds about multi turn chat.

10

HyDE

Have the model guess an answer first, then search using that guess.

THE PURPOSE

Questions and answers look different. A short question and a dense technical paragraph may not sit close together in embedding space even when one answers the other. HyDE, short for Hypothetical Document Embeddings, asks the model to write a fake answer, then searches with that fake answer instead. Answers match answers better than questions match answers.

WHEN TO USE IT

Short or jargon heavy queries against dense technical corpora. It is a specialist tool, not a default.

BACKGROUND

It is clever and genuinely works in the right conditions, but it adds a full generation before every search, which means latency and cost. Reach for hybrid search and reranking first. Knowing HyDE by name and knowing when not to use it is a stronger signal than deploying it everywhere, because it shows you evaluate techniques instead of collecting them.

HOW TO LEARN IT

Twenty lines on top of your existing retriever. Generate a hypothetical answer, embed that, search. Measure whether it beats your baseline. On many corpora it will not, and that null result is a legitimately good thing to be able to talk about.

IMPACT ● ● ○ ○ ○

Situational. Real gains in narrow cases.

HIRING SIGNAL ● ● ○ ○ ○

Bonus points, not a requirement.

11 Reranking

A second, smarter pass that reorders your search results.

THE PURPOSE

First stage retrieval is optimized for speed across millions of chunks, so it is a little blunt. A reranker takes the top 50 candidates and reads each one against the query properly, then reorders them. The best chunk moves from position 12 to position 1, where the model will actually pay attention to it.

WHEN TO USE IT

Almost always, once your baseline works. It is the highest quality per hour of effort upgrade in the entire stack.

BACKGROUND

The reason it works is architectural. Your embedding model encodes the query and the document separately, which is why it can precompute an index. A reranker, called a cross encoder, looks at query and document together, which is far more accurate and far too slow to run across everything. So you use the fast one to narrow and the slow one to sort. Retrieve 50, rerank, keep 5. Hosted options like Cohere Rerank are one API call, and open models run locally.

HOW TO LEARN IT

Add a reranker to a retriever you already have. Print the before and after ordering for ten queries side by side. It is the most satisfying diff in this entire guide.

IMPACT ● ● ● ● ●
Best effort to quality ratio available.

HIRING SIGNAL ● ● ● ● ○
Teams running RAG in production expect this.

12 Small to big retrieval

Search using tiny precise chunks, then send the larger surrounding section.

THE PURPOSE

Small chunks match well but lack context. Big chunks have context but match poorly. Small to big resolves the conflict: index the small pieces so search stays sharp, but when one hits, hand the model its parent section so the answer has room to breathe.

WHEN TO USE IT

Structured documents where surrounding context carries meaning. Legal contracts, technical manuals, API references, policy documents. Anywhere a sentence is meaningless without its heading.

BACKGROUND

You will also see this called parent document retrieval or auto merging retrieval. Implementation is a parent ID on every child chunk and a lookup after the search returns. If you have ever denormalized a table for read performance, this is the same instinct applied to text. It costs you nothing at query time beyond one extra fetch.

HOW TO LEARN IT

Chunk a technical doc at both sentence level and section level, store the mapping, and retrieve small while returning big. Compare answers against plain retrieval on questions that need surrounding context.

IMPACT ● ● ● ● ○
Big gains on structured content.

HIRING SIGNAL ● ● ● ○ ○
Marks you as someone who has hit the real failure modes.

13

Contextual retrieval

Adding a short summary of where a chunk came from before you embed it.

THE PURPOSE

A chunk reading “revenue grew 12 percent” is nearly useless on its own. Whose revenue, which quarter, which report. Before embedding, you prepend a sentence of context: “From the Q3 2024 earnings report, Northwind Systems, financial results section.” Now the chunk is findable and the model can interpret it.

WHEN TO USE IT

Large collections of similar looking documents. Financial filings, repeated report formats, ticket histories, meeting notes. Anywhere many chunks would otherwise be indistinguishable.

BACKGROUND

This is a newer technique that produced substantial retrieval improvements when Anthropic published on it, and it has been widely adopted since. The context line is usually generated once per chunk at indexing time by a cheap model, so it costs you an ingestion job, not runtime latency. It pairs naturally with hybrid search because the added text helps both keyword and semantic matching.

HOW TO LEARN IT

Take a folder of similar documents. Index once plainly, once with a generated context line prepended to each chunk. Query with something ambiguous like “what was the growth rate” and compare. The difference is not subtle.

IMPACT ● ● ● ● ○

Large gains on repetitive corpora.

HIRING SIGNAL ● ● ● ○ ○

Current. Shows you follow the field, not a 2023 tutorial.

14

Grounding and citations

Forcing answers to come from the retrieved text, and showing the source.

THE PURPOSE

A model asked a question it cannot answer will often invent something confident and wrong. Grounding means instructing it to answer only from the provided passages and to say it does not know otherwise.

Citations mean every claim points back to a document a human can open and verify.

WHEN TO USE IT

Every system with real users. It is the difference between a product a company can ship and a liability their legal team blocks. In regulated industries it is a hard requirement.

BACKGROUND

Part of this is prompt design and part is plumbing. You need chunk identifiers to survive the whole pipeline so the answer can reference them, and you need a UI that surfaces the source. The other half is handling the no answer case gracefully, which is genuinely hard because models are trained to be helpful and will stretch to fill a gap. A system that reliably says "I could not find this in your documentation" is more valuable than one that is right slightly more often but occasionally fabricates.

HOW TO LEARN IT

Number your chunks, require the model to cite by number, then write a validator that fails if a cited number was not actually retrieved. Now ask a question your documents cannot answer and confirm it declines instead of guessing.

IMPACT ● ● ● ● ●

Decides whether the system is trustworthy at all.

HIRING SIGNAL ● ● ● ● ●

Top tier concern for anyone shipping to customers.

15 Retrieval evaluation

Proving your system works instead of eyeballing a few answers.

THE PURPOSE

Without measurement, every change is a guess. Evaluation splits into two questions. Did retrieval find the right passages, and did the answer faithfully use them. You need both, because a perfect retriever feeding a sloppy prompt still produces a bad product.

WHEN TO USE IT

From the second week of any serious project. Build a small test set of real questions with known correct sources, and run it on every change like a test suite.

BACKGROUND

Two terms worth knowing. Recall at k asks whether the correct document appeared in your top k results at all, which isolates retrieval from

generation. Faithfulness asks whether the final answer is actually supported by the passages it was given. Fifty hand written question and source pairs beat any off the shelf benchmark, because they reflect your users. This is the concept engineers skip most often and the one that separates people who tinker from people who get promoted, because it turns opinion into evidence.

HOW TO LEARN IT

Write 20 questions against your own documents and record which chunk should be retrieved for each. Compute recall at 5. Now change one thing, your chunk size, and rerun. You now have the ability to say "this change improved retrieval by 14 percent," which is the sentence that gets people hired.

IMPACT ● ● ● ● ●

Makes every other improvement on this list measurable.

HIRING SIGNAL ● ● ● ● ●

The single strongest differentiator in interviews.

WHAT TO DO WITH THIS

Do not study all 15. Build one thing that uses 8 of them.

Pick a corpus you actually care about: your team's documentation, a framework you use, a folder of PDFs you keep rereading. Chunk it by structure. Embed it. Put it in pgvector. Add hybrid search and a reranker. Write 20 evaluation questions and measure recall at 5. Make it cite sources and admit when it does not know.

That project touches nearly everything in this guide, takes a couple of weekends, and gives you something to talk about for forty five minutes in an interview. **The engineers getting hired are not the ones who memorized the list. They are the ones who shipped the thing.**

+ The Signal Project framework for building portfolio work that hiring managers actually respond to
+ Feedback on your retrieval pipeline from engineers who have shipped one
+ Mentors from Amazon, Spotify, and other teams running this in production
+ The Get Hired system: positioning, resume, and interview prep built for the pivot

JOIN THE COMMUNITY

You are unspecialized, not behind.

https://www.skool.com/baswe-ai